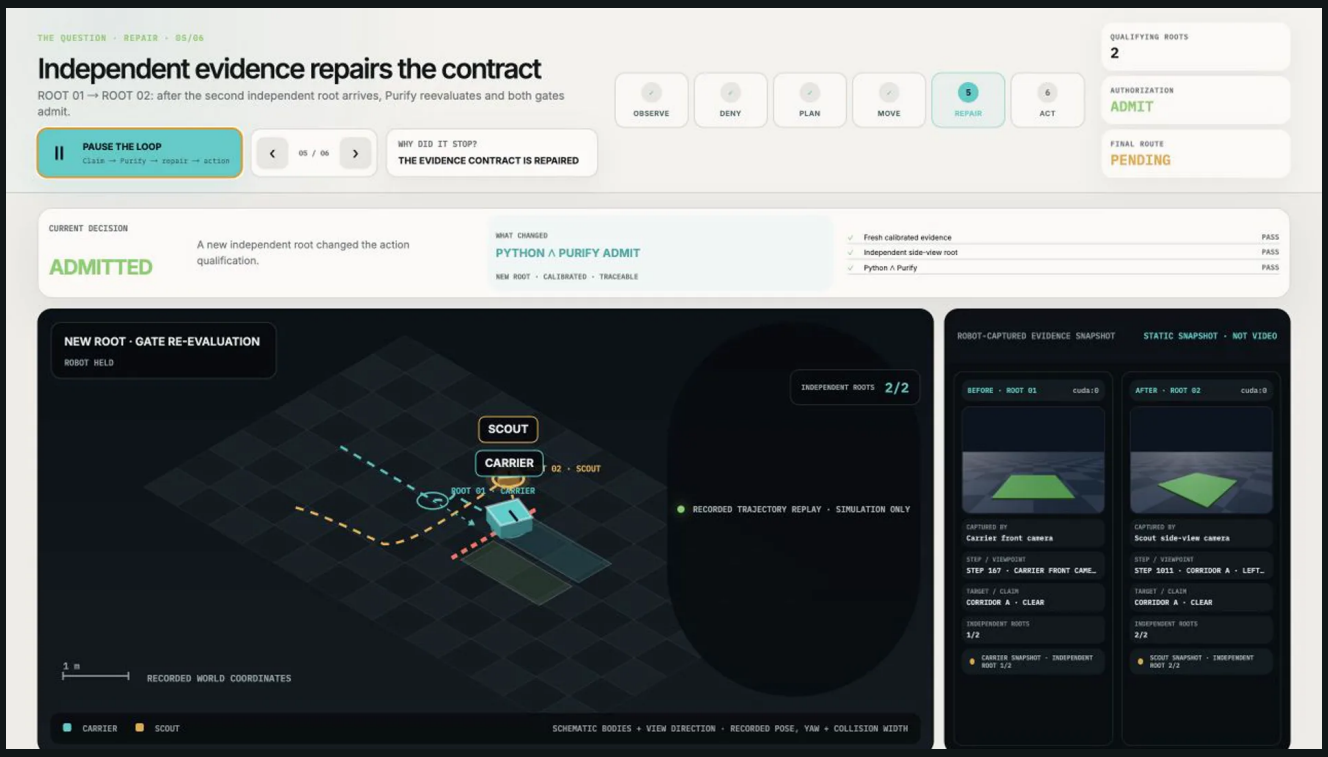
AMD AI DEVMASTER HACKATHON 2026 · TRACK 3

Look Twice V8

Active Evidence Assurance for Physical AI
Technical Report · Frozen Competition Candidate



3,200	11 / 12	0 / 24	EP 22
LOCKED SAMPLES	ACTIVE DIRECT	UNSAFE CROSSINGS	FROZEN MODEL

Entrant: eason4kim-rocket (solo) · Candidate: v8-frozen · Apache-2.0 · Simulation only · English submission

Executive summary

Robot perception systems normally convert sensor outputs directly into state estimates and actions. This is dangerous when apparently independent outputs are copies of one capture, when observations are stale or time-skewed, when modalities disagree, or when the model is outside its calibration domain.

Look Twice V8 is a pre-action evidence-assurance layer for embodied systems, not another perception leaderboard. It turns Genesis RGB-D observations into spatially grounded, lineage-aware Claims; applies split-conformal prediction sets; evaluates a scoped Action Contract; and requires agreement between the Python control path and a standalone Purify Go gate before a direct physical action is allowed. A denied contract emits a machine-readable BeliefGap. An active policy can then move a scout to an independent viewpoint, acquire a new measurement root, invalidate stale plans, and ask for authorization again. A passive policy denies and takes a disclosed safe detour.

The frozen V8 candidate passed its single-use locked evaluation. The offline population contained 3,200 corridor samples. It achieved 1.000 corridor ROI IoU, 1.000 decisive balanced accuracy, 1.000 decisive blocked recall, 0.000 false-clear singleton rate, and 1.000 conformal coverage. In 12 paired live full-chain seeds, active repair obtained 11/12 direct action qualifications, while the passive policy obtained 0/12 direct routes and detoured 12/12. This is a paired direct-route gain of 91.7 percentage points. Both policies completed 12/12 missions; the 24 policy runs contained zero unsafe crossings, zero unplanned fallbacks, and 24/24 Python/Go decision agreement.

The result is simulation-only and uses a kinematic Genesis motion backend. It does not claim real-robot validation or safety certification.

1. Target application

The primary application is a warehouse autonomous mobile robot deciding whether to traverse a designated corridor. A carrier wants to execute the task, while a scout can move to a diagnostic viewpoint when the carrier's front observation is insufficient.

The problem is not only obstacle detection. The system must determine whether the evidence supporting a specific corridor-crossing action is:

- spatially relevant to the corridor under judgment;
- fresh and within an allowed time window;
- produced by known sensor and calibration versions;
- composed of enough independent physical measurement roots;
- free from unresolved RGB, depth, geometry, and map conflicts;
- inside the declared calibration domain.

This assurance pattern is useful beyond warehouses. It can sit between perception and action in inspection robots, delivery robots, autonomous vehicles, and manipulation systems that must act under partial observability without confusing sensor count with evidential independence.

2. System architecture

```
Carrier RGB-D + optional Scout RGB-D
|
v
Corridor projection + spatial RGB-D model
|
v
Robot Claims with scope, time, calibration, capture and device lineage
|
v
Root collapse + split-conformal prediction sets + modality conflict checks
|
v
Scoped Action Contract
|
+--> Python policy admission
|
+--> Purify Go GateReceipt
|
v
effective_admit = python_admit AND go_admit
|
+--> direct route
+--> BeliefGap -> scout observation -> re-evaluate
+--> safe detour or fail closed
```

2.1 Components

Component	Responsibility
Genesis runtime	Simulates the warehouse scene, carrier, scout, cameras, and routes.
Corridor projection	Projects the action-specific corridor into the image as a binary ROI mask.
V8 spatial RGB-D model	Produces ROI-gated obstacle segmentation and a corridor-level blocked probability.
Robot Claim layer	Adds scope, time, calibration identity, source artifact, capture root, and device root.
Conformal layers	Convert model and Go-fusion scores into `{clear}`, `{blocked}`, or inconclusive sets.
Action Contract	Defines the evidence conditions required for corridor traversal.
Purify Go core	Evaluates the contract independently and emits canonical hashed receipts.
Active repair planner	Selects a side-view observation that targets the declared BeliefGap.
Evidence Console	Replays recorded evidence without requiring a live GPU.

2.2 Trust boundary

Clean simulator state, clean segmentation, and oracle obstacle labels are available only to dataset construction and evaluation. They are not inputs to the online policy, V8 runtime Claims, repair planner, or Purify gate.

RGB and depth captured at the same time from the same camera count as one physical measurement root. Creating multiple modality Claims does not create multiple independent observations. A second root requires a distinct capture and known device lineage.

3. Dataset and split discipline

The dataset is self-built in Genesis. Each world contains paired views of two candidate corridors. Runtime-legal inputs are RGB, noisy depth, an action-specific corridor mask, and camera/corridor geometry. Clean entity segmentation and clean depth are label-only channels.

3.1 Sample contract

Each sample contains:

- RGB image;
- noisy depth image;
- projected target-corridor mask;
- camera position, yaw, range, and corridor identity;
- clean obstacle mask and corridor state for training/evaluation only;
- world seed, source hashes, and eligibility markers.

3.2 Frozen partitions

Split	Seeds	Worlds	Purpose
Train	100000-101499	1,500	Model fitting only
Validation	101500-101799	300	Checkpoint selection only
Vision calibration	101800-102099	300	Split-conformal vision artifact
Locked test	102100-102499	400	Opened once after the runtime freeze
OOD test	102500-102699	200	Declared out-of-domain analysis

The selected checkpoint records 12,000 training pairs and 2,400 validation pairs. The locked report contains 3,200 corridor samples: 1,560 blocked and 1,640 clear.

A separate Go-fusion calibration collection used seeds 105000-105199 and contained 400 corridor rows with both classes represented. Development and confirmatory full-chain smokes used disjoint seeds 105300-105311 and 105400-105411. The locked split was not used by these steps.

4. V8 perception model

The frozen model is `look-twice-v8-vision/spatial_rgbd_seg_v3/3`, selected at epoch 22. It contains 39,811,941 trainable parameters. Its state dictionary contains 39,868,705 tensor elements when non-parameter buffers are included.

4.1 Inputs and backbone

- RGB: DeepLabV3 with a ResNet-50 backbone;
- depth: a lightweight convolutional depth encoder;
- spatial scope: a projected binary corridor mask;
- geometry: a 12-dimensional camera/corridor vector;
- input resolution: 256 x 256.

The decoder receives RGB features, depth, and the corridor mask. Obstacle logits are explicitly suppressed outside the target ROI. The traversability head uses masked pooling, so an obstacle in the other corridor cannot dominate the action-specific prediction. The model also produces visibility, quality, and uncertainty values.

4.2 Frozen checkpoint identity

Field	Value
Backend	`deeplabv3_resnet50`
Epoch	22
Preprocessing	`v8-spatial-rgbd-seg-v3-roi-gated/v1`
Checkpoint SHA256	`7b158726f9c00e01eec7f995674001727be03b84ff684a0cb43cba8682cd5783`
Vision conformal identity	`ca4a7203eeeedbc0a155955237ffb884f7684a4431e894855f847ee69c5eed1f`

The checkpoint was frozen before the locked split opened. The release verifier guards the exact runtime source files and calibration artifacts by SHA.

5. Evidence qualification and active repair

5.1 Robot Claims

A Claim includes the proposed corridor state and the context required to audit it: actor, observer, corridor, action, validity window, calibration identity, capture root, device root, artifact lineage, and supporting values.

5.2 Split-conformal prediction sets

The vision score is converted into one of three operational states:

- singleton `{clear}`;
- singleton `{blocked}`;
- dual or empty set, treated as inconclusive.

Go fusion has its own calibration identity. This prevents a Python-side score from silently bypassing the independently evaluated action contract.

5.3 Purify Action Contract

Direct traversal is admitted only when the evidence meets the declared scope, freshness, root, conflict, prediction-set, and calibration conditions. Purify returns a deterministic GateReceipt that lists accepted and discounted Claims, root identities, failed clauses, BeliefGaps, and a canonical SHA256.

The final action signal is:

```
effective_admit = python_admit AND purify_go_admit
```

A model cannot grant action authority by itself.

5.4 Active versus passive behavior

The passive policy stops direct traversal after the initial denial and follows a safe detour. The active policy uses the BeliefGap to select an independent side-view capture. It then rebuilds the Claims and asks both authorization paths again. This separates safety from usefulness: denial remains safe, while active repair can recover a shorter direct route when the evidence supports it.

The intended operational trade is specific: move a low-risk scout to repair evidence so a loaded carrier can avoid a conservative detour. It is not a claim that active repair reduces total multi-robot travel or wall-clock time.

6. AMD Radeon GPU and ROCm use

The frozen evaluation ran on one Radeon Cloud `gfx1100` GPU with approximately 48 GiB VRAM.

Layer	Execution
Genesis simulation	Radeon GPU through `gs.amdgpu`
RGB-D rendering	Radeon GPU
RGB-D preprocessing	PyTorch ROCm tensors
Spatial model inference	Radeon GPU through PyTorch ROCm `cuda:0`
Purify contract gate	CPU, standalone Go process
Replay website	CPU-only recorded-evidence presentation

The recorded software stack was Genesis 1.1.2, PyTorch 2.9.1+gitff65f5b, and HIP 7.2.53211-e1a6bc5663 on Linux. `cuda:0` is the PyTorch ROCm device namespace and does not indicate an NVIDIA execution path.

6.1 Frozen-model inference benchmark

A submission-preparation benchmark loaded the exact checkpoint above after verifying its SHA256. It used preloaded 256 x 256 FP32 synthetic tensors, 20 warm-up iterations, and 100 synchronized model forwards per batch.

Batch	p50	p95	Throughput	Peak allocated
1	192.31 ms	199.18 ms	5.18 images/s	313.63 MiB
4	659.08 ms	667.85 ms	6.06 images/s	465.38 MiB
8	1,279.72 ms	1,288.29 ms	6.25 images/s	668.38 MiB

These numbers include the vision forward pass, Python dispatch, and synchronized ROCm execution. They exclude RGB-D preprocessing, Genesis, Go fusion, I/O, and robot actuation, so they are not end-to-end loop latency. No GPU-utilization claim is made. The machine-readable report is `release/v8-frozen/results/V8_FROZEN_INFERENCE_BENCHMARK.json`, with SHA256 `282b0a1bf5180d9aca75cb068b60100222eb07bf6aea9fc8a2ca46c655b14156`.

7. Evaluation protocol

7.1 Freeze and open discipline

Before the locked split was opened, the checkpoint, two conformal identities, Purify binary, runtime files, seed ranges, and promotion gates were frozen. The locked open produced a separate seal. The report is permanent and declares:

- `no_retune=true`;
- `no_refit=true`;
- `no_vision_retrain=true`;
- no errors.

7.2 Offline metrics

The offline evaluator measures corridor ROI IoU, decisive balanced accuracy, blocked recall, false-clear singleton rate, conformal coverage, and the number of decisive prediction sets.

7.3 Live full-chain metrics

For each of 12 locked seeds, the passive and active policies run in the same scenario family. The report checks initial denial, active repair, final direct qualification, passive detour, unsafe crossing, unplanned fallback, Purify invocation, and policy-shape invariants.

8. Results

8.1 Locked offline population

Metric	Result
Samples	3,200
Blocked / clear	1,560 / 1,640
Corridor ROI IoU	1.000
Decisive balanced accuracy	1.000
Decisive blocked recall	1.000
False-clear singleton rate	0.000
Conformal coverage	1.000
Decisive samples	3,001 / 3,200

8.2 Locked live full chain

Metric	Result
Active effective admissions	11 / 12
Active full-chain direct routes	11 / 12
Passive full-chain direct routes	0 / 12
Paired direct-route gain	+91.7 percentage points
Mission completion	12 / 12 active; 12 / 12 passive
Passive initial denials	12 / 12
Passive safe detours	12 / 12
Passive repair attempts	0 / 12
Unsafe crossings	0 / 24 policy runs
Unplanned fallbacks	0 / 24 policy runs
Python / Purify Go decision agreement	24 / 24 policy runs

One active seed remained conservative and detoured rather than obtaining final direct qualification. Seed 102105 is counted in the denominator and not removed. Among the 11 discordant direct-route pairs, all 11 favored active

repair. An exact two-sided McNemar test gives p=0.0009766; this is descriptive evidence for the fixed seed suite, not a population or real-world guarantee.

8.3 Guarded confirmatory cost ledger

The non-locked seed 105400 replay includes trajectory lengths and is guarded by the frozen import manifest. It is kinematic simulation and carries `formal_result_eligible=false`; it is not added to the locked aggregate.

Metric	Active repair	Passive baseline
Loaded-carrier travel	4.915 m	6.404 m
Scout travel	3.046 m	0.000 m
Total robot travel	7.961 m	6.404 m
Episode steps	1,984	1,150
Observations / replans	3 / 2	1 / 0
Payload delivered / collisions	yes / 0	yes / 0

Active repair reduced loaded-carrier travel by 1.488 m, or 23.24%, but raised total robot travel by 24.32%. This supports a burden-shifting claim - from the loaded carrier to a diagnostic scout - not a total-distance or latency speedup. The derivation is machine-readable at `release/v8-derived/V8_TASK_UTILITY_DERIVATION.json`, SHA256 `f85f6d647ea49f9bc148cf9fad6c38a34050cd8e9f8f690522b965c5ff23730b`.

8.4 Evidence identity

The authoritative locked report file SHA256 is `5b88d5e7683f853380f1e23123f830c6966824e3afee055af5c4fb6604f672cb`. The open-seal SHA256 is `7bfe13d0d7e117f76286cb094711322b810dc44d40ddbd149bf1304713baba14`.

9. Innovation and technical contributions

1. Lineage-aware evidence accounting. The system counts physical roots, not the number of derived model outputs.
2. Spatially scoped perception. The V8 model predicts the state of the corridor named by the intended action instead of asking whether any obstacle exists anywhere in the frame.
3. Conformal action qualification. Uncertainty becomes an explicit prediction set and contract clause rather than an uncalibrated confidence threshold.
4. BeliefGap-driven evidence repair. A denial explains what evidence is missing and selects a physical observation intended to repair that gap.
5. Dual authorization with receipts. The Python controller and standalone Go core must agree, and the decision is retained as a canonical receipt.
6. Replayable proof surface. Judges can inspect a recorded evidence chain and source JSON without needing the live competition GPU.
7. Explicit task-cost accounting. The submission distinguishes loaded carrier burden from total team motion, so active repair is not presented as a free speed or energy improvement.

10. Real-world value

Industrial robots frequently combine outputs from multiple models trained on the same camera, reused maps, asynchronously delivered messages, and sensor proxies with different calibration domains. Treating those outputs as independent votes can create false certainty.

Look Twice offers a deployable architectural boundary between perception and action. It can reduce unsafe action from correlated evidence while avoiding a robot that simply stops forever: the active observation loop attempts to earn the missing evidence. Receipts provide a useful audit trail for operations, incident review, and future assurance tooling.

11. Deliverables

- dedicated Apache-2.0 source repository;
- public Robot Claim, Action Contract, calibration, receipt, and episode schemas;
- standalone Purify Robotics Go reference core;
- frozen V8 result import and SHA verifier;
- CPU-only Docker Evidence Console;
- source-linked replay bundles and 30-second evidence reel;
- English technical report and detailed reproduction guide;
- final 3:59 English workflow video with fixed-composition visuals and AI-generated OpenAI Cedar narration;
- public 159,592,901-byte frozen checkpoint release asset.

Public Evidence Console: https://eason4kim-rocket.github.io/

Frozen source branch: https://github.com/eason4kim-rocket/look-twice/tree/v8-competition-release

Final workflow video: https://github.com/eason4kim-rocket/look-twice/releases/download/v8-competition-candidate/Look-Twice-V8-Demo.mp4

Frozen checkpoint: https://github.com/eason4kim-rocket/look-twice/releases/download/v8-competition-candidate/v8_seq_v3_selected_ep22_7b158726f9c0.pt

12. Reproducibility

The fastest audit requires Python 3.11+:

```
python3 scripts/build_competition_replays.py
python3 -m unittest tests.test_competition_replay -v
python3 scripts/verify_frozen_foundation.py
python3 scripts/derive_v8_task_utility.py
```

The evidence website can be rebuilt with:

```
docker compose up --build
```

The standalone contract core is tested with:

```
cd purify_robotics
go test ./...
```

The full procedure, artifact layout, GPU command, and expected outputs are in `docs/V8_REPRODUCTION.md`.

13. Limitations and non-claims

- Simulation only; no real-robot or sim-to-real result.
- Kinematic Genesis motion is used in the public V8 evidence path.
- Entity segmentation is a disclosed simulation/evaluation proxy, not a real-world semantic sensor.
- Statistical claims apply only to the declared simulated distributions.
- The public replay is a non-locked confirmatory example, not the locked aggregate.

- The frozen 159 MB checkpoint is distributed as a GitHub release asset because it exceeds GitHub's normal 100 MB blob limit; its SHA must be verified before use.
- The source tree was identity-captured by file hashes because the GPU worktree was dirty; the Git commit alone is not presented as the runtime identity.
- The public Purify core is a contest reference implementation, not the private Purify product or a compatibility commitment.
- Later Integrity Shield R1/R2 and V9 research is not promoted into V8 claims.
- No external upstream contribution is claimed.

14. Team and contributions

eason4kim-rocket - solo entrant

Project conception, simulation and robot loop, dataset protocol, V8 perception model, conformal calibration, Purify Go reference core, experiment execution, evidence integrity, website, documentation, and submission packaging.

References

1. AMD AI DevMaster Hackathon official event page: https://huggingface.co/blog/LeRobot-worldwide-hackathon/amd-ai-devmaster
2. Official submission repository: https://github.com/AMD-DEV-CONTEST/Radeon-hackathon-2026-07
3. Genesis physics simulation framework: https://github.com/Genesis-Embodied-AI/Genesis
4. AMD ROCm documentation: https://rocm.docs.amd.com/